Lecture Notes: Feature Encoding

CIC1205 - Machine Learning

Eduardo Bezerra

April 6, 2026

0 Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Types of Categorical Variables . . . 3
1.3 Encoding and Data Leakage . . . 3
1.4 Running Example: the Banking Dataset . . . 4
1.5 Overview of Encoding Methods . . . 4

2 Label Encoding **5**
2.1 Concept . . . 5
2.2 When to Use Label Encoding . . . 6
2.3 Implementation . . . 6

3 One-Hot Encoding (Dummy Encoding) **7**
3.1 Concept . . . 7
3.2 Procedure . . . 8
3.3 Implementation . . . 8
3.4 Advantages and Limitations . . . 11

4 Ordinal Encoding **11**
4.1 Concept . . . 11
4.2 Procedure . . . 12
4.3 Examples . . . 12
4.4 Advantages and Limitations . . . 13

5 Frequency Encoding **15**
5.1 Concept . . . 15
5.2 When to Use Frequency Encoding . . . 15
5.3 Implementation . . . 16
5.4 Advantages and Limitations . . . 17

6 Target Encoding **17**
6.1 Concept . . . 17
6.2 Risk of Data Leakage . . . 18
6.3 Implementation . . . 19
6.4 Advantages and Limitations . . . 21

7 Handling unseen categories at inference time **22**

8 Feature Encoding in the Model Selection Pipeline **23**
8.1 Where does encoding fit? . . . 23
8.2 Correct workflow . . . 23
8.3 Why encoding must be inside the pipeline . . . 23
8.4 Encoding as a hyperparameter . . . 24
8.5 Implementation with pipelines . . . 24

9 Summary and Practical Guidance **25**
9.1 Comparative Overview . . . 25

9.2 Decision Guide 25
9.3 The Pipeline Principle 26
9.4 Encoding in the Context of Model Selection 27
9.5 Looking Ahead: Embeddings 27

1 Introduction

1.1 Motivation

Most machine learning algorithms operate exclusively on numerical inputs. Real-world datasets, however, routinely contain **categorical features**: variables whose values are labels or categories rather than numbers. Examples include the job title of a bank customer, the marital status of a patient, or the color of a product. Before such features can be fed into a learning algorithm, they must be converted into a numerical representation. This conversion process is called **feature encoding**.

Feature Encoding

Feature encoding is the process of transforming categorical variables into numerical representations suitable for use as input to machine learning algorithms.

The choice of encoding has a direct impact on model quality. A poorly chosen encoding can introduce artificial ordering relationships between categories that do not exist in the data. For example, encoding colors as `Red=1`, `Blue=2`, `Green=3` implicitly tells the model that Green > Blue > Red, which is semantically meaningless. Conversely, a well-chosen encoding preserves the semantic structure of the original variable while keeping the representation compact and informative.

1.2 Types of Categorical Variables

Categorical variables come in two fundamentally different kinds, and the appropriate encoding strategy depends on which kind is at hand. A useful guiding question is: *does it make sense to say that one category is "greater than" another?* If yes, the variable is ordinal; if not, it is nominal.

Nominal variables. Categories have no meaningful order. Examples: job title (`management`, `technician`, `entrepreneur`), color (`Red`, `Blue`, `Green`), contact type (`cellular`, `telephone`, `unknown`). Imposing a numeric order on nominal categories is semantically incorrect and may harm model performance.

Ordinal variables. Categories have a natural order, but the distances between consecutive categories are not necessarily equal or known. Examples: education level (`primary` < `secondary` < `tertiary`), T-shirt size (`Small` < `Medium` < `Large`). Encoding ordinal variables without respecting their order loses potentially valuable information.

Variable type	Order meaningful?	Example
Nominal	No	job, color, contact type
Ordinal	Yes	education level, size (S/M/L)

1.3 Encoding and Data Leakage

Regardless of the encoding method used, a critical principle must be observed: the encoding must be fitted only on training data and then applied to both training and test data.

Computing encoding statistics (e.g., category frequencies or target means) on the full dataset before splitting introduces *data leakage*: information from the test set influences the transformation applied to the training set, leading to overly optimistic performance estimates.

To make this concrete: if frequency encoding is fitted on the full dataset and the test set contains many rare categories, their frequencies are estimated using test-set observations: the model implicitly "sees" the test distribution during training. This concern is especially relevant for target encoding and frequency encoding, where the transformation depends on the data distribution.

Pipeline rule for encoding

Always fit the encoder on the training portion of the data only. When using `scikit-learn`, wrap encoders inside a `Pipeline` or `ColumnTransformer` so that `fit_transform` is called only on training folds and `transform` is called on validation and test folds.

1.4 Running Example: the Banking Dataset

The examples throughout these notes use the `banking` dataset. Each row corresponds to a bank customer and contains both numerical features (e.g., age, balance, duration of the last call) and categorical features. The target variable `y` indicates whether the customer subscribed to a term deposit (`yes`/`no`).

Selected categorical columns and their unique values are shown below.

Column	Unique values
`job`	`management, technician, entrepreneur, blue-collar, unknown, retired, admin., services, self-employed, unemployed, housemaid, student`
`marital`	`married, single, divorced`
`education`	`tertiary, secondary, unknown, primary`
`contact`	`unknown, cellular, telephone`
`month`	`jan, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec`
`poutcome`	`unknown, failure, other, success`

This dataset illustrates the full spectrum of encoding challenges: some variables are nominal (`job`, `contact`), some are naturally ordinal (`education`), and some have high cardinality (`job`, `month`).

1.5 Overview of Encoding Methods

These notes cover five main encoding methods, each suited to different situations. Note that the best choice also depends on the type of model being used: linear models are sensitive to the numeric ordering implied by an encoding, whereas tree-based models split on threshold values and are generally more robust to arbitrary integer assignments.

Method	Best for	Handles high cardinality?	Uses target?
Label Encoding	Target variable only	**Yes**	No
One-Hot Encoding	Nominal, few categories	**No**	No
Ordinal Encoding	Ordinal features	**Yes**	No
Frequency Encoding	High-cardinality nominal	**Yes**	No
Target Encoding	High-cardinality + strong correlation to target	**Yes**	**Yes (leakage risk)**

Encoding and model type

The choice of encoding interacts with the choice of model. The table below summarises common pairings:

Model family	Recommended encoding
Linear models (Logistic Regression, Linear SVM)	One-Hot Encoding
Tree-based models (Random Forest, Gradient Boosting)	Frequency or Target Encoding
Neural networks	One-Hot Encoding or learned embeddings

These are guidelines, not hard rules. Cross-validation is the ultimate arbiter.

2 Label Encoding

2.1 Concept

Label encoding converts each category into a unique integer. Given a categorical variable with k distinct values, the encoder assigns integers $0, 1, \ldots, k-1$ to the categories.

Label Encoding

Label encoding maps each unique category of a variable to a distinct integer. The assignment is arbitrary: no ordering relationship among the integers is implied or guaranteed to be meaningful.

Example. The target variable `y` of the banking dataset takes values `yes` and `no`. After label encoding, `no` maps to `0` and `yes` maps to `1`.

2.2 When to Use Label Encoding

Correct usage of Label Encoding

In scikit-learn, `LabelEncoder` is designed and documented for encoding the target variable y. For input features, integer encoding is generally unsafe for nominal variables used with linear models: the model will interpret the integers as a meaningful ordering and magnitude, which does not exist.
For example, encoding `job` as `admin=0`, `blue-collar=1`, `entrepreneur=2`, ... and feeding these values into a linear regression implies that `entrepreneur` is numerically "twice" `blue-collar`, which is meaningless.
Tree-based models are less sensitive to this problem because they split on thresholds rather than treating values as a continuous scale. Even so, using an appropriate encoding from the outset is good practice.
For input features, use one-hot encoding, ordinal encoding, or another method described in the following sections.

2.3 Implementation

The code below uses `LabelEncoder` from scikit-learn to encode the binary target column `y` of the banking dataset, then splits the data into training and test sets.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

X = df_banking_train.drop(columns=['y'])
y = df_banking_train['y']

print(y.tail())
# 45206    yes
# 45207    yes
# 45208    yes
# 45209     no
# 45210     no
# Name: y, dtype: object

label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)

print(y[-5:])
# [1 1 1 0 0]
# no -> 0, yes -> 1  (alphabetical order: 'no' < 'yes')

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

Does encoding the target before splitting cause data leakage?

A common question is whether applying `LabelEncoder` to the target variable `y` *before* calling `train_test_split` introduces data leakage. The short answer is no, and understanding why helps clarify what leakage actually is.

Data leakage occurs when information from the test set influences the training process, causing the model to appear better than it truly is on unseen data. The key word is *information*: leakage is harmful when the training pipeline learns something from the test examples that it would not have access to in production.

`LabelEncoder` applied to the target simply re-codes class labels according to a fixed, deterministic rule (alphabetical order in this case: `"no"` $\rightarrow$ 0, `"yes"` $\rightarrow$ 1). This rule does not depend on the data in any statistical sense—it would produce exactly the same mapping regardless of which examples end up in the training set and which end up in the test set. Encoding before or after the split yields identical results.

The intuition above breaks down, however, as soon as encoding is applied to the **features** rather than the target. Consider the following cases:

- **Target encoding** (replacing a categorical feature with the mean of the target within each category): fitting this encoder on the full dataset before splitting leaks target information from the test set into the feature values used during training, a clear case of leakage.
- **Standardisation or PCA** applied to numerical features: fitting the scaler or the decomposition on the full dataset exposes test-set statistics (mean, variance, principal components) to the training procedure, which is also leakage, albeit often milder in practice.

The general rule is: **any transformation whose parameters are estimated from the data must be fitted on the training set only**, and then applied (without re-fitting) to the test set. This is precisely what `sklearn`'s `Pipeline` is designed to enforce. Encoding the target with `LabelEncoder` is safe before the split because its "parameters" are just the alphabet, not statistics derived from your data.

Checking the mapping

`LabelEncoder` assigns integers alphabetically by default. Always verify the mapping with `label_encoder.classes_` to confirm which integer corresponds to which category, especially for binary targets where `1` should represent the positive class (`yes`, `success`, etc.).

Entry Ticket 1

A practitioner applies `LabelEncoder` to the feature `job` in the banking dataset and feeds the result directly into a linear regression model.

1. What implicit assumption does this encoding introduce?
2. Why is this assumption problematic for a nominal variable like `job`?
3. Which encoding would be more appropriate, and why?
4. Would the same problem arise if a decision tree were used instead? Explain.

3 One-Hot Encoding (Dummy Encoding)

3.1 Concept

One-hot encoding (also called *dummy encoding*) represents a categorical variable with k distinct categories by creating k new binary columns, one per category. For each

observation, the column corresponding to its category is set to 1; all other columns are set to 0.

One-Hot Encoding

Given a nominal variable X with categories $\{c_1, c_2, \ldots, c_k\}$, one-hot encoding creates k binary indicator variables $X_{c_1}, X_{c_2}, \ldots, X_{c_k}$, where

$$X_{c_j} = \begin{cases} 1 & \text{if } X = c_j, \\ 0 & \text{otherwise.} \end{cases}$$

Intuition. One-hot encoding makes the absence/presence of each category explicit. No artificial ordering is introduced, which makes it the natural choice for nominal variables with a small number of categories.

3.2 Procedure

The steps of one-hot encoding are:

1. Identify the categorical variable to be encoded.
2. Create one binary indicator column for each unique category.
3. For each observation, set the column corresponding to its category to 1 and all other columns to 0.
4. Optionally, drop one of the k columns to avoid the *dummy variable trap* (multicollinearity), retaining $k - 1$ columns instead. The dropped category becomes the *reference category*, and all other columns are interpreted relative to it.

Dummy variable trap. When all k indicator columns are included together, their sum is always 1 for every observation. This creates perfect linear dependence among the columns, which can cause numerical instability in some models. Dropping one column removes this redundancy; the dropped category is then implicitly captured by the intercept.

Dummy variable trap and model type

The dummy variable trap is a concern primarily for **linear models** (e.g., linear/logistic regression), where perfect multicollinearity makes the design matrix non-invertible. **Tree-based models** are unaffected: they split on individual columns one at a time and do not invert any matrix. It is nevertheless good practice to use `drop_first=True` consistently to keep the feature space compact.

Figure 1 illustrates the one-hot encoding of the feature `Color` with three categories: `Red`, `Blue`, and `Green`.

3.3 Implementation

In pandas, `pd.get_dummies` applies one-hot encoding to a column. Setting `drop_first=True` removes one column to avoid the dummy variable trap.

Color
Red
Blue
Green
Red
Green

one-hot →

Red	Blue	Green
1	0	0
0	1	0
0	0	1
1	0	0
0	0	1

Figure 1: One-hot encoding of the `Color` feature. Each category is represented by a separate binary column. For each row, exactly one column equals 1.

Binary variable (two categories). The original variable `Gender` has two categories: `Male` and `Female`. Applying one-hot encoding would normally create two binary columns, one for each category. However, with `drop_first=True`, one of the categories is removed to avoid redundancy.

In this case, the column `Female` is dropped and becomes the *reference category*. The remaining column, `Male`, indicates whether each observation belongs to that category:

- `True` (or 1) means the observation is `Male`;
- `False` (or 0) means the observation is `Female`.

Thus, a single binary column is sufficient to represent the original variable without loss of information. This is a special case of one-hot encoding where $k = 2$, so only $k - 1 = 1$ column is needed.

Interpretation

When `drop_first=True` is used, the omitted category is always interpreted as the baseline. All remaining columns indicate deviations from this reference.

```python
import pandas as pd

data = {'Gender': ['Male', 'Female', 'Male', 'Female', 'Male']}
df = pd.DataFrame(data)

# drop_first=True keeps k-1 = 1 column (Male), dropping Female as
    reference
binary_encoded = pd.get_dummies(df['Gender'], drop_first=True)
print(binary_encoded)
#      Male
# 0    True
# 1   False
# 2    True
# 3   False
# 4    True
```

Nominal variable with three categories. The variable `Color` has three categories: `Red`, `Blue`, and `Green`. Applying one-hot encoding creates one binary column for each

category. Each column indicates the presence (1 or `True`) or absence (0 or `False`) of that category in a given observation.

For each row, exactly one of the columns takes the value `True`, while the others are `False`. For example:

- The first observation is `Red`, so the column `Red` is `True`, while `Blue` and `Green` are `False`.
- The second observation is `Blue`, so only the column `Blue` is `True`.

This encoding introduces no artificial ordering among categories: each color is represented independently. The resulting representation is therefore suitable for nominal variables, where categories have no intrinsic ranking.

Full vs. reduced encoding

In this example, all $k = 3$ columns are kept. This is often convenient for interpretation. However, for models sensitive to multicollinearity (such as linear regression), it is common to drop one column and keep only $k - 1$ columns, using the omitted category as a reference.

```
data = {'Color': ['Red', 'Blue', 'Green', 'Red', 'Green']}
df = pd.DataFrame(data)

one_hot_encoded = pd.get_dummies(df['Color'])
print(one_hot_encoded)
#      Blue   Green    Red
# 0   False   False   True
# 1    True   False  False
# 2   False    True  False
# 3   False   False   True
# 4   False    True  False
```

Applied to the banking dataset. The `contact` column has three categories: `unknown`, `cellular`, and `telephone`. Applying `get_dummies` with `drop_first=True` produces two indicator columns.

```
temp = df_banking_train.copy()
print(temp.contact.unique())
# ['unknown' 'cellular' 'telephone']

X_train_encoded = pd.get_dummies(temp['contact'], drop_first=True)
print(X_train_encoded)
#         telephone  unknown
# 0           False     True
# 1           False     True
# ...
# 45209        True    False
# 45210       False    False
# [45211 rows x 2 columns]
```

3.4 Advantages and Limitations

Advantages. No artificial ordering is imposed. Compatible with all learning algorithms, including linear models and neural networks.

Limitations. Creates k (or $k-1$) new columns per categorical variable. For high-cardinality variables (many unique categories, such as `job` with 12 values or `month` with 12 values), this leads to a large number of sparse features and can increase computational cost and memory usage significantly. Alternative encodings should be considered in those cases.

Common mistake: one-hot encoding high-cardinality variables

Applying one-hot encoding to a variable with hundreds of categories can severely increase dimensionality and harm model performance. For example, a dataset with 50,000 rows and 20 categorical columns each with 30 categories would add $20 \times 30 = 600$ new columns, most of which are nearly always zero. For high-cardinality variables, prefer frequency encoding or target encoding.

Cardinality threshold

As a practical guideline, one-hot encoding works well when a categorical variable has fewer than 10–15 unique categories. This threshold is not a hard rule: it depends on dataset size and the model being used. Beyond that range, frequency encoding or target encoding are typically more efficient choices.

Entry Ticket 2

The `job` column in the banking dataset has 12 unique values.

1. How many binary columns would one-hot encoding create (without `drop_first`)? How many with `drop_first=True`?
2. What is the dummy variable trap and why can it be problematic?
3. For a dataset with 50 000 rows and 20 categorical columns, each with 30 categories on average, how many total new columns would one-hot encoding add?
4. For which type of model is the dummy variable trap more concerning: a logistic regression or a random forest? Why?

4 Ordinal Encoding

4.1 Concept

Ordinal encoding assigns integer values to the categories of an ordinal variable according to their natural order. Unlike label encoding, which assigns integers arbitrarily, ordinal encoding respects a meaningful ranking specified by the practitioner.

Ordinal Encoding

Given an ordinal variable X with ordered categories $c_1 \prec c_2 \prec \cdots \prec c_k$, ordinal encoding maps each category to a distinct integer that preserves the order:

$$c_1 \mapsto 0, \quad c_2 \mapsto 1, \quad \ldots, \quad c_k \mapsto k-1.$$

The mapping must be defined explicitly by the practitioner, not inferred automatically, to ensure the ordering is semantically correct.

4.2 Procedure

1. Identify the ordinal variable and define the complete ordered list of its categories, from the smallest to the largest value.
2. Map each category to its position in the ordered list.
3. Replace the original column with the resulting integer column.

Key distinction from label encoding. Label encoding arbitrarily assigns integers based on the order in which categories are encountered (e.g., alphabetically), which may not correspond to any meaningful semantic order. Ordinal encoding requires the practitioner to explicitly specify the correct order, making it semantically meaningful for ordinal variables. Using label encoding on an ordinal variable will usually produce an incorrect ordering.

4.3 Examples

T-shirt sizes. The variable `Size` has categories ordered as `Small` $\prec$ `Medium` $\prec$ `Large`. Using a pandas mapping dictionary:

```
import pandas as pd

data = {'Size': ['Small', 'Medium', 'Large', 'Medium', 'Small']}
df = pd.DataFrame(data)

size_mapping = {'Small': 1, 'Medium': 2, 'Large': 3}
df['Size_encoded'] = df['Size'].map(size_mapping)

print(df)
#       Size  Size_encoded
# 0    Small             1
# 1   Medium             2
# 2    Large             3
# 3   Medium             2
# 4    Small             1
```

Multiple ordinal columns with scikit-learn. The `OrdinalEncoder` class from scikit-learn encodes multiple ordinal columns simultaneously. The key parameter is `categories`, which receives a list of ordered category lists, one per column.

```
from sklearn.compose import ColumnTransformer
```

```python
from sklearn.preprocessing import OrdinalEncoder

selected_columns = ['marital', 'month', 'education']

# Define the correct semantic order for each column
categories_order = [
    ["single", "married", "divorced"],  # marital
    ["jan", "feb", "mar", "apr", "may", "jun",
     "jul", "aug", "sep", "oct", "nov", "dec"],  # month
    ["unknown", "primary", "secondary", "tertiary"]  # education
]

ct = ColumnTransformer(
    [("enc", OrdinalEncoder(categories=categories_order),
      selected_columns)],
    remainder="passthrough"
)
X_train_encoded = ct.fit_transform(X_train)

# Example output (first 5 rows, selected columns):
# marital  month  education
#     0.0    5.0        3.0
#     1.0   10.0        2.0
#     1.0    5.0        2.0
#     2.0    9.0        3.0
#     1.0    4.0        2.0
```

In the result above, `education = 3.0` corresponds to `tertiary` (the highest level), and `education = 2.0` to `secondary`, correctly reflecting the semantic ordering.

4.4 Advantages and Limitations

Advantages. Compact representation (one integer per ordinal column). Preserves the semantic ordering of categories, which can improve model performance when the ordering genuinely correlates with the target variable.

Limitations. Applicable only to truly ordinal variables. If applied to a nominal variable, the implied numeric ordering is meaningless and may mislead the model.

Furthermore, the integer representation implies uniform distances between consecutive categories (e.g., that the difference from `primary` to `secondary` is the same as from `secondary` to `tertiary`). This assumption affects **linear models**, which treat the encoded integer as a continuous input; a model might learn, for instance, that moving up one education level increases the predicted probability by a fixed amount. **Tree-based models** are less affected, because they only compare values against a threshold (e.g., `education` ≥ 2.0) and do not assume linearity.

`LabelEncoder` vs. `OrdinalEncoder`

A natural follow-up question is how `LabelEncoder` and `OrdinalEncoder` differ. In terms of the encoding logic, they are equivalent: both assign an integer to each category according to alphabetical order. The distinction is practical and concerns the **shape of the input they expect**.

- `LabelEncoder` expects a **1-D array**, a single vector of labels. This is why it

is the natural choice for encoding the target variable `y`.

- `OrdinalEncoder` expects a **2-D array**, a matrix with one column per categorical feature. This makes it compatible with the feature matrix `X` and, crucially, with `sklearn`'s `Pipeline` and `ColumnTransformer`.

A concrete consequence: if you attempt to use `LabelEncoder` inside a `Pipeline` to encode a feature column, `sklearn` will raise an error because the transformer receives a 2-D slice of `X` rather than the 1-D input it expects. `OrdinalEncoder` handles this correctly.

As a practical rule of thumb: use `LabelEncoder` for the target `y`, and `OrdinalEncoder` (inside a `ColumnTransformer` or `Pipeline`) for categorical features in `X`.

Entry Ticket 3

A practitioner needs to encode the `education` column of the banking dataset, whose values are `unknown`, `primary`, `secondary`, and `tertiary`.

1. Is `education` a nominal or ordinal variable? Justify your answer.
2. What happens if `LabelEncoder` is used instead of `OrdinalEncoder`? What risk does this create?
3. Write the `categories_order` list you would pass to `OrdinalEncoder` for this column. Explain your choice of ordering.
4. Why is the uniform-distance assumption of ordinal encoding more problematic for a linear model than for a decision tree?

Which encoder to use for ordinal classification?

In ordinal classification, the target variable has a natural ordering among its categories (e.g., `low` < `medium` < `high`). This raises an important question: can `LabelEncoder` or `OrdinalEncoder` be used safely in this setting?

The answer is: **not without care**. Both encoders assign integers according to alphabetical order by default, which may or may not coincide with the true semantic order of the classes. For example, alphabetical order would encode `high` → 0, `low` → 1, `medium` → 2, the opposite of the intended ordering. Relying on the default behaviour is therefore fragile and error-prone.

Encoding ordinal features in `X`. `OrdinalEncoder` accepts an explicit `categories` argument that lets you specify the desired order:

```
from sklearn.preprocessing import OrdinalEncoder

enc = OrdinalEncoder(categories=[['low', 'medium', 'high']])
X_encoded = enc.fit_transform(X[['severity']])
# low -> 0, medium -> 1, high -> 2
```

This is the recommended approach when ordinal features appear in `X`, as it integrates cleanly with `Pipeline` and `ColumnTransformer`.

Encoding an ordinal target `y`. `sklearn` does not provide a dedicated encoder for ordinal targets. The simplest and most explicit approach is a manual mapping:

```
order = {'low': 0, 'medium': 1, 'high': 2}
y = y.map(order)
```

This makes the intended ordering immediately visible in the code and leaves no room for silent alphabetical mis-ordering.

A broader note. Encoding the target as ordered integers is a necessary first step, but it does not by itself make a classifier respect the ordinal structure during training. Standard classifiers (e.g., logistic regression, random forests) treat the encoded integers as arbitrary class labels and ignore the ordering. Truly exploiting the ordinal structure requires either a dedicated ordinal loss function or a specialised algorithm, a topic we will return to later in the course.

5 Frequency Encoding

5.1 Concept

Frequency encoding replaces each category with a numerical value derived from how often that category appears in the training data. The most common variant uses the **relative frequency** (proportion of occurrences), though absolute counts are also used.

Frequency Encoding

Given a categorical variable X and the training set $\mathcal{D}_{\text{train}}$, frequency encoding maps each category c to its relative frequency:

$$\phi(c) = \frac{\#\{x_i = c \mid x_i \in \mathcal{D}_{\text{train}}\}}{|\mathcal{D}_{\text{train}}|}.$$

The mapping ϕ is estimated on the training set and applied to both training and test sets.

The encoded value of a category thus captures how common that category is in the data. Rare categories receive small values; frequent categories receive large values.

5.2 When to Use Frequency Encoding

Frequency encoding is a good default for situations where:

- The categorical variable has high cardinality (many unique categories), making one-hot encoding impractical.
- A tree-based model is used (e.g., Random Forest, Gradient Boosting), because frequency-based splits can be informative when the frequency of a category correlates with the target variable.
- No target variable is available (unsupervised settings), since frequency encoding does not require y.

Caveat: collisions

If two different categories happen to have the same frequency, frequency encoding assigns them the same numeric value and the model cannot distinguish them. For example, if both `Red` and `Green` appear 40% of the time, they both receive the value 0.4 and become indistinguishable to the model. This is a known limitation of the method. One practical mitigation is to combine frequency encoding with another feature (e.g., the category itself as an ordinal code) when collisions are suspected to matter.

5.3 Implementation

Simple example. The code below demonstrates a simple implementation of frequency encoding. The goal is to replace each category in the feature `Color` with its relative frequency (proportion) in the dataset. First, a DataFrame is created with a single categorical column. Then, the instruction `df['Color'].value_counts(normalize=True)` computes the normalized frequency of each category, that is, the proportion of times each color appears in the data. For example, "Red" and "Green" each occur in 2 out of 5 instances (0.4), while "Blue" appears once (0.2). These frequencies can then be used as numerical representations of the categories. An important caveat is illustrated in this example: different categories may share the same frequency (a *collision*), meaning that they become indistinguishable after encoding. While frequency encoding is simple and efficient, such collisions may lead to loss of information depending on the task.

```
import pandas as pd

data = {'Color': ['Red', 'Blue', 'Green', 'Red', 'Green']}
df = pd.DataFrame(data)

frequency_encoded = df['Color'].value_counts(normalize=True)
print(frequency_encoded)
# Color
# Red       0.4
# Green     0.4
# Blue      0.2
# Name: proportion, dtype: float64
# Note: Red and Green have the same frequency (collision).
```

Reusable encoder compatible with scikit-learn pipelines. To integrate frequency encoding into a scikit-learn `Pipeline` and respect the train/test split, a custom transformer is needed. The `FrequencyEncoder` class below inherits from `BaseEstimator` and `TransformerMixin`, following scikit-learn's API. The frequency map is estimated only during `fit` (on training data) and applied during `transform`.

```
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

class FrequencyEncoder(BaseEstimator, TransformerMixin):
    def __init__(self, columns=None):
        self.columns = columns
        self.frequency_maps = {}
```

```python
    def fit(self, X, y=None):
        """Compute frequency of each category in specified columns."""
        if self.columns is None:
            self.columns = X.select_dtypes(
                include=['object', 'category']
            ).columns
        for col in self.columns:
            self.frequency_maps[col] = X[col].value_counts().to_dict()
        return self

    def transform(self, X):
        """Apply frequency encoding to specified columns."""
        X = X.copy()
        for col in self.columns:
            # fillna(0) handles categories not seen at training time
            X[col] = X[col].map(self.frequency_maps[col]).fillna(0)
        return X

selected_columns = ['job', 'marital', 'month']

pipeline = Pipeline([
    ('freq_encoder', FrequencyEncoder(columns=selected_columns)),
    ('classifier', RandomForestClassifier(
        n_estimators=100, random_state=42))
])

pipeline.fit(X_train[selected_columns], y_train)
y_pred = pipeline.predict(X_test[selected_columns])
```

The `fit`/`transform` separation ensures that frequency maps computed on the training set are applied to the test set without leakage. Unseen categories at test time receive a frequency of 0.

5.4 Advantages and Limitations

Advantages. Produces a single feature per categorical column regardless of cardinality, so it scales well to variables with many categories. No target variable is required, making it applicable in unsupervised settings.

Limitations. Category frequency may not correlate with the target variable, in which case the encoded values carry little useful information. Identical frequencies for distinct categories (collisions) cannot be resolved by the encoder alone.

6 Target Encoding

6.1 Concept

Target encoding, also called *mean target encoding* (or simply *mean encoding*), is both powerful and dangerous. It replaces each category with a statistic of the target variable y computed over all training examples that belong to that category. This technique is widely used in practice, particularly in machine learning competitions. `sklearn` provides it via `TargetEncoder` (available from version 1.3 onwards). Used correctly, it can be

one of the most informative encodings available; used carelessly, it introduces subtle data leakage that inflates performance estimates.

Target Encoding

Given a categorical variable X and a target variable y, target encoding maps each category c to the mean of y for all training examples with $X = c$:

$$\tau(c) = \mathbb{E}[y \mid X = c] \approx \frac{1}{|\{i : x_i = c\}|} \sum_{\{i : x_i = c\}} y_i.$$

The intuition is that categories with a high mean of the target variable receive a high encoded value, and vice versa. This directly embeds the relationship between the feature and the target into the encoding, making it potentially very informative.

6.2 Risk of Data Leakage

The most important concern with target encoding is data leakage. If the encoding statistics are computed on the same data used for training (or, worse, on the full dataset including the test set), the model can "see" the target variable through the encoded feature, which inflates performance estimates.

How leakage occurs. Suppose category c appears only once in the training set, in example i with target y_i. The target mean for category c is then exactly y_i. When the model is trained on this example, its encoded feature already contains the answer, making the learning task trivially easy for that example. The resulting model will be overfit and will generalize poorly.

Mitigations.

- **Leave-one-out encoding:** compute the category mean excluding the current example's own label. This eliminates the most direct form of leakage, where a singleton category would otherwise expose its label exactly, However, it is computationally expensive for large datasets and can produce noisy estimates, since each example's encoded value is computed from a slightly different set of rows.
- **Smoothing (shrinkage):** blend the category mean with the global mean of y using a weighting that depends on the number of observations in the category:

  $$\hat{\mu}_c = \frac{n\,\bar{y}_c + \lambda\,\bar{y}}{n + \lambda},$$

  where $\lambda > 0$ controls the strength of regularisation. Categories with few observations are pulled strongly toward the global mean, reducing overfitting; large categories are barely affected.
- **Cross-validation-based encoding (cross-fitting):** divide the training set into k folds. For each fold, compute the category means using only the other $k - 1$ folds, then use those means to encode the held-out fold. Repeating this for all folds ensures that no example's label contributes to its own encoded value, the same principle that motivates nested cross-validation. This approach is also known as

out-of-fold encoding in the machine learning competition literature, and is the default strategy in `sklearn`'s `TargetEncoder`.

Leave-one-out encoding (item 1) is a special case of this procedure with $k = n$: each "fold" contains a single example. In practice, a moderate value such as $k = 5$ or $k = 10$ offers a better bias–variance trade-off, since the category means are estimated from a larger and more stable subset of the data.

6.3 Implementation

Manual implementation. The code snippet below illustrates a simple manual implementation of target encoding for a categorical variable in a regression setting. The idea is to replace each category in the feature `City` with the average value of the target variable (`Salary`) observed for that category. First, a DataFrame is created with cities and their corresponding salaries. Then, the instruction `df.groupby('City')['Salary'].mean()` computes the mean salary for each city, producing a mapping such as "New York → 82500", "Chicago → 71000", and so on. Next, the `map` function is used to replace each city in the original column with its corresponding mean value, creating a new numerical feature `City_encoded`. This transformation converts a categorical variable into a continuous one that captures information about the relationship between the category and the target, which can be useful for models that cannot directly handle categorical inputs.

```
import pandas as pd

# Regression example: encode City using mean of Salary
data = {'City': ['New York', 'Los Angeles', 'Chicago',
                 'New York', 'Chicago'],
        'Salary': [80000, 75000, 70000, 85000, 72000]}
df = pd.DataFrame(data)

city_means = df.groupby('City')['Salary'].mean()
df['City_encoded'] = df['City'].map(city_means)

print(df)
#            City  Salary  City_encoded
# 0      New York   80000       82500.0
# 1  Los Angeles   75000       75000.0
# 2       Chicago   70000       71000.0
# 3      New York   85000       82500.0
# 4       Chicago   72000       71000.0
```

Target encoding for classification

The target encoding example above uses a continuous target (salary), where each category is replaced by the mean of the target within that category. A natural question is: what changes when the target is discrete, as in a classification problem?

The mechanics are the same. For a binary target encoded as 0/1, the mean within a category is simply the **proportion of positive examples** in that category. If, for instance, 3 out of 4 customers from "New York" made a purchase ($y = 1$), the encoded value for that city would be $3/4 = 0.75$. The formula is identical to

the regression case; the only difference is that the output is now interpretable as an empirical probability.

```
import pandas as pd

data = {'City':      ['New York', 'Los Angeles', 'Chicago',
                      'New York', 'Chicago', 'New York', 'New York
    '],
         'Purchased': [1, 0, 1, 1, 0, 1, 0]}
df = pd.DataFrame(data)

city_means = df.groupby('City')['Purchased'].mean()
df['City_encoded'] = df['City'].map(city_means)
print(df)
#             City  Purchased  City_encoded
# 0      New York          1      0.750000
# 1  Los Angeles           0      0.000000
# 2       Chicago          1      0.500000
# 3      New York          1      0.750000
# 4       Chicago          0      0.500000
# 5      New York          1      0.750000
# 6      New York          0      0.750000
```

Leakage is an even sharper concern in classification. In the naive implementation above, the encoded value for each example is computed using *all* rows in its category, including the example itself. For a binary target this means the model can partially recover the true label of each training instance directly from the encoded feature—a form of leakage that is especially damaging when a category contains very few examples. In the extreme case of a singleton category, the encoded value equals the label exactly.

Two standard remedies are used in practice:

- **Smoothing.** Shrink the category mean toward the global positive rate $\bar{y}$ by a factor that depends on the category size n:

$$\hat{\mu}_c = \frac{n\,\bar{y}_c + \lambda\,\bar{y}}{n + \lambda},$$

  where $\lambda > 0$ controls the strength of regularisation. Small categories are pulled strongly toward the global mean; large categories are barely affected.
- **Cross-fitting (out-of-fold encoding).** Split the training set into k folds. For each fold, compute the category means using only the *other* $k-1$ folds, then apply them to encode the held-out fold. This ensures that no example's label contributes to its own encoded value. `sklearn`'s `TargetEncoder` implements this strategy by default.

As with any transformation that estimates statistics from the target, **the encoder must be fitted on the training set only** and then applied without re-fitting to the validation and test sets.

Robust implementation with the `category_encoders` library. The `TargetEncoder` class from the `category_encoders` library provides a more robust implementation that includes smoothing. The encoder follows the scikit-learn API (`fit`/`transform`), making

it straightforward to integrate into pipelines. It is fit on the training set and applied to the test set, preventing leakage.

```
import pandas as pd
from category_encoders import TargetEncoder

# Prepare training and test sets
df = df_banking_train.copy()
X_train = df.drop(columns=['y'])
y_train = df['y'].replace({'no': 0, 'yes': 1})  # no=0, yes=1 (
    standard)

df = df_banking_test.copy()
X_test = df.drop(columns=['y'])
y_test = df['y'].replace({'no': 0, 'yes': 1})

# Encode 'job' and 'marital' using target encoding
encoder = TargetEncoder(cols=['job', 'marital'])

# Fit ONLY on training data, then apply to both sets
X_train_encoded = encoder.fit_transform(X_train, y_train)
X_test_encoded  = encoder.transform(X_test)
```

In the encoded training data, the `job` column is replaced by a floating-point value between 0 and 1 reflecting the mean target value for each job category. For example, `management` might receive 0.138, reflecting the proportion of management customers who subscribed to the term deposit (where $y = 1$ means `yes`).

6.4 Advantages and Limitations

Advantages. Directly captures the relationship between the categorical feature and the target variable. Produces a single feature per categorical column regardless of cardinality. Often very effective for high-cardinality variables in gradient-boosted tree models.

Limitations. Requires the target variable, so it cannot be used in unsupervised settings. Prone to data leakage if not implemented carefully. Can overfit when a category has very few examples in the training set, which is mitigated by smoothing.

Entry Ticket 4

A practitioner wants to apply target encoding to the `job` column of the banking dataset.

1. Why is it incorrect to compute the target mean for each job category using the entire training+test dataset?
2. A category `student` appears only 3 times in the training set. Two of those students subscribed. Without smoothing, what value would target encoding assign to `student`? Why might this be unreliable?
3. How does smoothing (shrinkage toward the global mean) address the problem in the previous question? Intuitively, what does it "assume" about categories with very few observations?

7 Handling unseen categories at inference time

A subtle but important question is: what should an encoder do when it encounters a category at inference time that was never seen during `fit`? This is sometimes referred to as an **out-of-distribution (OOD) category** problem, and it is a common source of silent failures in production systems.

The answer depends on the encoder being used.

OrdinalEncoder and LabelEncoder. Both encoders maintain a fixed vocabulary of categories seen during `fit`. By default, `sklearn` raises an error when an unknown category is encountered at inference time. `OrdinalEncoder` exposes a `handle_unknown` parameter that allows a more graceful fallback:

```
from sklearn.preprocessing import OrdinalEncoder

enc = OrdinalEncoder(
    handle_unknown='use_encoded_value',
    unknown_value=-1    # assigned to any unseen category
)
```

The choice of `unknown_value` is a modelling decision. A value of -1 signals "unknown" to the model, but whether the model handles it sensibly depends on the algorithm.

OneHotEncoder. Setting `handle_unknown='ignore'` causes all dummy columns for an unseen category to be set to zero. This is equivalent to saying "this example provides no information about any known category", which is a reasonable and interpretable default in most cases.

```
from sklearn.preprocessing import OneHotEncoder

enc = OneHotEncoder(handle_unknown='ignore')
```

Target encoding. If a category was not seen during `fit`, no target mean is available for it. The natural fallback is the **global mean of** y computed on the training set, which conveys "we have no category-specific information, so we revert to the baseline rate." This is consistent with the smoothing formula: as the number of observations $n \to 0$, the smoothed estimate $\hat{\mu}_c = (n\bar{y}_c + \lambda\bar{y})/(n + \lambda)$ collapses to the global mean $\bar{y}$. `sklearn`'s `TargetEncoder` implements this behaviour automatically.

The general principle. Every preprocessing step that learns a vocabulary or statistics from the training set must have an explicit policy for inputs that fall outside that vocabulary. Failing to define such a policy is a common source of silent errors in production: the pipeline may not raise an exception, but the encoded values it produces for unseen categories are meaningless, and the model's predictions for those examples will be unreliable. Handling unknown categories should therefore be treated as a first-class design decision, not an afterthought.

8 Feature Encoding in the Model Selection Pipeline

8.1 Where does encoding fit?

Recall from the previous lecture that model selection evaluates entire pipelines — and encoding is a component of that pipeline. Feature encoding is not a preliminary step performed once before model training. Instead, it is part of the learning pipeline and must be treated as such.

In modern machine learning workflows, the pipeline typically includes:

- preprocessing (e.g., encoding, scaling, imputation),
- the learning algorithm,
- and possibly post-processing steps.

From the perspective of model selection, the entire pipeline is the object being evaluated.

Key Result

Encoding is part of the learning pipeline. Different encoding choices produce different models and must be evaluated within the model selection process.

8.2 Correct workflow

The correct procedure follows the same logic as any hyperparameter selection:

1. Split the data into development set and test set.
2. Within the development set:
   - For each candidate pipeline (encoding + model):
     (a) Fit the encoder *only* on the training fold. This step learns data-dependent statistics (e.g., category frequencies, target means) exclusively from training data.
     (b) Transform the training data using the fitted encoder.
     (c) Apply the *same* transformation to the validation fold.
     (d) Train and evaluate the model.
3. Select the best pipeline using validation or cross-validation.
4. Retrain the entire pipeline on the full development set.
5. Evaluate once on the test set.

8.3 Why encoding must be inside the pipeline

This problem is known as data leakage: if encoding is applied *before* the data split, information from held-out data (validation or test sets) may contaminate the training process. This is especially problematic for:

- frequency encoding,

- target encoding,
- any transformation that depends on data distribution.

Data leakage example

If frequency encoding is computed on the full dataset before splitting, the frequency counts reflect categories in the validation and test folds. The model indirectly "sees" held-out data before evaluation, leading to an optimistic bias in performance estimates.

8.4 Encoding as a hyperparameter

The choice of encoding method should be treated as a hyperparameter of the pipeline. To isolate the effect of encoding, one can fix the learning algorithm and vary only the encoding method:

- One-Hot Encoding + Logistic Regression
- Ordinal Encoding + Logistic Regression
- Target Encoding + Logistic Regression

More generally, the search space may combine encoding methods *and* learning algorithms simultaneously, defining a larger grid of candidate pipelines:

- One-Hot Encoding + Logistic Regression
- Frequency Encoding + Random Forest
- Target Encoding + Gradient Boosting

In either case, each combination defines a distinct candidate model and must be evaluated using the same validation strategy.

Practical rule

Do not choose the encoding method using the test set. Encoding must be selected using validation or cross-validation, just like any other hyperparameter.

8.5 Implementation with pipelines

In `scikit-learn`, this is achieved using `Pipeline` or `ColumnTransformer`, which guarantee that:

- `fit` is called only on training folds,
- `transform` is applied to validation and test folds,
- and the entire pipeline is evaluated consistently across splits.

A minimal example combining `ColumnTransformer` and `Pipeline` inside `cross_val_score` is shown below.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
```

```python
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

encoder = ColumnTransformer([
    ("ohe", OneHotEncoder(handle_unknown="ignore"), cat_features)
], remainder="passthrough")

pipeline = Pipeline([
    ("encoding", encoder),
    ("model",    LogisticRegression())
])

# fit() is called only on each training fold;
# transform() is applied to each validation fold automatically
scores = cross_val_score(pipeline, X_dev, y_dev, cv=5)
```

Mental model

Think of the pipeline as a single function:

$$\hat{y} = f(\mathbf{x}_{\text{raw}})$$

where f encapsulates both encoding and learning. Selecting a pipeline means selecting f as a whole — there is no meaningful separation between "choosing an encoder" and "choosing a model."

9 Summary and Practical Guidance

9.1 Comparative Overview

The encoding methods covered in these notes differ in their assumptions, computational cost, and suitability for different types of categorical variables. Table 1 summarises the key dimensions.

9.2 Decision Guide

Before encoding any categorical variable, ask yourself these three questions:

1. **Is there a natural order?** If yes, ordinal encoding preserves it.
2. **How many categories does the variable have?** Few categories favour one-hot encoding; many categories favour frequency or target encoding.
3. **Does the variable correlate strongly with the target?** If yes and cardinality is high, target encoding (with smoothing) may offer the best signal.

The following decision path formalises these questions:

1. **Is this the target variable?** → Use label encoding.
2. **Is the variable ordinal (does category order matter)?** → Use ordinal encoding, with the order specified explicitly.

Table 1: Comparison of feature encoding methods.

Method	**Best for**	**High cardinality?**	**Main risk / caveat**
Label Encoding	Target variable y only	**Yes**	Meaningless for input features (especially linear models)
One-Hot Encoding	Nominal, few categories	**No**	Feature space explosion
Ordinal Encoding	Ordinal features	**Yes**	Wrong order $\Rightarrow$ incorrect encoding; uniform-distance assumption matters for linear models
Frequency Encoding	High-cardinality nominal; tree models	**Yes**	Collisions (same frequency)
Target Encoding	High-cardinality + strong target correlation	**Yes**	Data leakage if fitted on full data

3. **Is the variable nominal with few categories ($\lesssim$15)?** $\rightarrow$ Use one-hot encoding.
4. **Is the variable nominal with many categories?**
   - If a tree-based model is used, frequency encoding is a practical choice.
   - If the variable has a strong relationship with the target, target encoding (with smoothing and proper train/test separation) may improve performance further.

9.3 The Pipeline Principle

In all cases where the encoding involves any computation on the data (frequency counts, target means, ordinal positions inferred from data), the encoder must be treated as a learned transformation and fitted only on training data. The correct workflow is always:

1. Fit the encoder on $\mathcal{D}_{\text{train}}$.
2. Apply `transform` to $\mathcal{D}_{\text{train}}$ to produce encoded training features.
3. Apply `transform` to $\mathcal{D}_{\text{val}}$ and $\mathcal{D}_{\text{test}}$ using the parameters estimated in step 1.

This principle extends to cross-validation: the encoder must be re-fitted inside each training fold, not on the full development set. In scikit-learn, wrapping encoders in a `Pipeline` object enforces this correctly.

Golden rule of feature encoding

Any encoding that learns from the data must be fitted exclusively on the training portion. Encoding the full dataset before splitting is a form of data leakage.

9.4 Encoding in the Context of Model Selection

Feature encoding is a preprocessing step, and its choice is itself a **hyperparameter** of the machine learning pipeline. Different encoding strategies may yield different model performance, and the best strategy should be chosen via validation (cross-validation or a held-out validation set), not by inspecting test performance.

This means that encoding decisions belong inside the model selection loop: for each candidate encoding, the entire pipeline (encoding + model) should be evaluated via the chosen validation strategy. The test set must remain sealed throughout this process, exactly as described in the companion notes on model selection.

9.5 Looking Ahead: Embeddings

The encodings discussed in these notes represent categories as hand-crafted numerical values. An alternative approach, common in **deep learning**, is to let the model learn the representation automatically. In this paradigm, each category is mapped to a dense low-dimensional vector called an **embedding**. Unlike one-hot encoding, embeddings can capture semantic similarity between categories: two job titles that tend to appear in similar contexts will have similar embedding vectors. Embeddings are the standard approach for categorical variables in neural networks and will be covered in the deep learning portion of the course.

9 References

- Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1), 27–32. https://dl.acm.org/doi/10.1145/507533.507538
- Category Encoders library documentation: https://contrib.scikit-learn.org/category_encoders/
- scikit-learn preprocessing documentation: https://scikit-learn.org/stable/modules/preprocessing.html